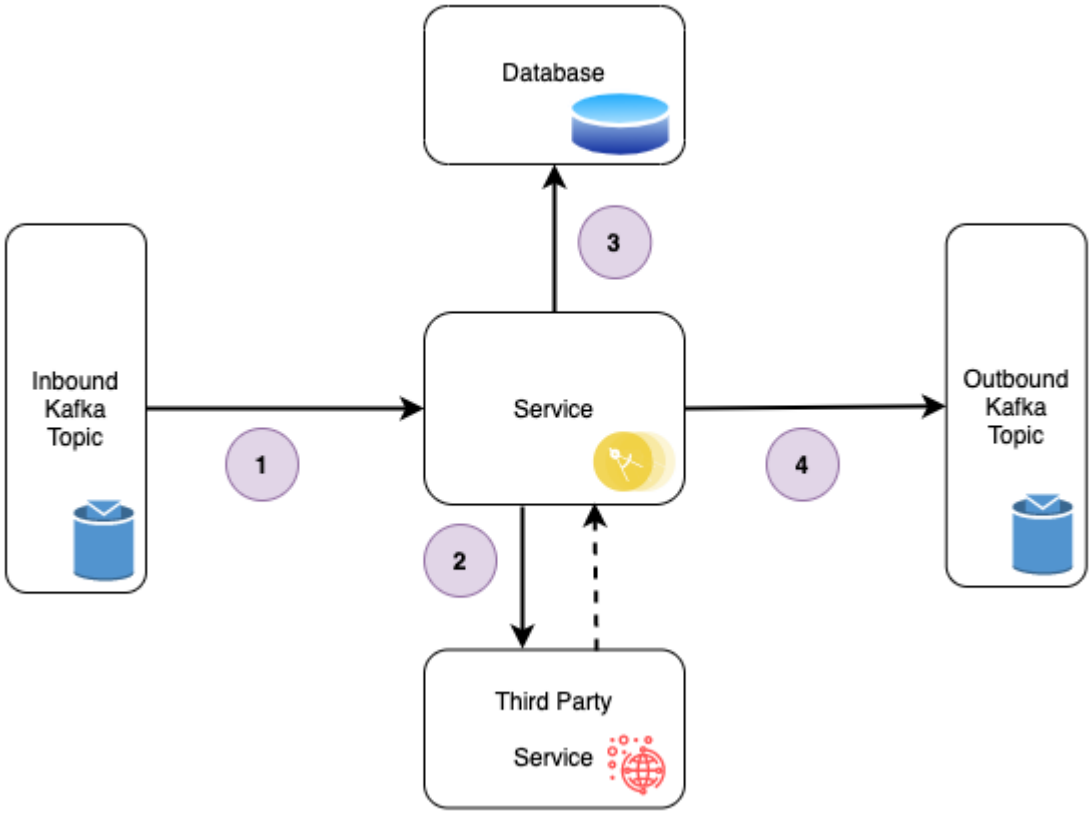
Introduction

Managing duplicate events is an inevitable aspect of distributed messaging using Kafka. This article looks at the patterns that can be applied in order to deduplicate these events.

Duplicate Events

Consider the following flow where a message is consumed from a topic, triggering a REST POST call to a third party service, a database INSERT to create a new record, and the publishing of a resulting event to an outbound topic.



1. Service consumes a message from Kafka.

2. Service makes a REST POST request to a third party service.

3. Service INSERTs a record in the database.

4. Service publishes a resulting event to Kafka.

A failure could happen at any stage in this flow. The flow must be implemented to not mark the message as consumed until the processing has completed. This ensures that upon failure the message will remain on the topic and be polled by another consumer in the consumer group. The deduplication patterns that are applied will then determine what duplicates will occur, whether a duplicate POST, duplicate database INSERT, or duplicate outbound event.

Deduplication Patterns

Patterns that cater for duplicate messages:

1. Idempotent Consumer Pattern

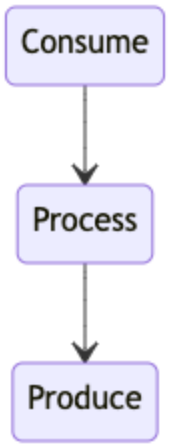
- Track received message IDs in the database.
- Use a locking flush strategy to stop duplicates being processed until message ID is committed or rolled back.
- Upon successful commit any duplicate awaiting lock is discarded.
- Message ID commit happens in the same transaction as any other database writes, making these actions atomic.

2. Transactional Outbox - messages published to outbox table with CDC

- Outbound message committed to the outbox table in DB in the same transaction as any other database writes, making these actions atomic.
- Kafka Connect CDC (change data capture) publishes messages to Kafka outbound topic.

3. Kafka Transaction API - for exactly-once delivery semantics

- Kafka's exactly-once semantics guarantees that the three consume, process and produce steps will happen exactly once.



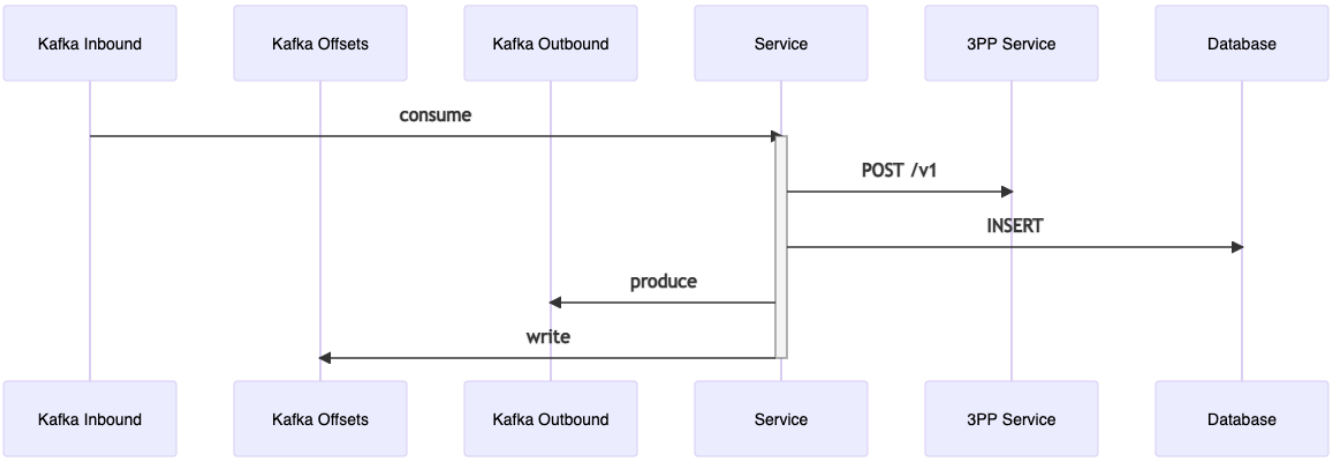
- It is important to understand that the consume and process steps could however happen multiple times.
- Uses the transactional log as a centralised, single source of truth for all ongoing transactions, guaranteeing it is atomic.

It is also possible to combine the Idempotent Consumer pattern with the Transactional Outbox pattern.

The Database transactions and Kafka transactions cannot be committed atomically. While a chained transaction manager can be used to commit these using a two-phase commit, this does not guarantee both transactions will complete atomically, and resources can be left in an inconsistent state. This means it is not possible to combine the Idempotent Consumer pattern with Kafka Transactions pattern without risking data loss.

Applying The Patterns

This sequence diagram describes the flow illustrated at the top of this article.



Only once the resulting event is published is the consumer offsets Kafka topic written to. This marks the message as consumed, and it will not be re-polled by another consumer in the same consumer group. If the consumer offsets are not written due to a preceding failure, then the message will be re-polled.

The following table summaries what duplicate actions (POST request, database INSERT, outbound event PRODUCE) can occur at different failure points based on which patterns have been applied.

	Happy Path	Service Dies After POST & INSERT	Service Dies After Produce	Service Dies After DB Transaction Committed	Consume Times Out
No Deduplication Pattern	None	POST INSERT	POST INSERT PRODUCE	N/A	POST INSERT PRODUCE
Idempotent Consumer	None	POST	POST PRODUCE	None	None
Transactional Outbox	None	POST	N/A	POST INSERT PRODUCE	POST INSERT PRODUCE
Idempotent Consumer & Transactional Outbox	None	POST	N/A	None	None
Kafka Transactions	None	POST	POST INSERT	N/A	POST INSERT

Some scenarios are not applicable to every pattern as the failure points can differ based on what patterns are in place. Each scenario for each pattern is diagrammed in the second part of this article.

The 'Consume Times Out' scenario refers to where a consumer event poll does not complete before the timeout, so the message is redelivered to a second consumer instance as the Kafka Broker believes the original consumer may have died.

Recommendations

Implementing the Idempotent Consumer and Transactional Outbox patterns together is the recommended approach to minimise duplicate actions occurring. The only risk here is a duplicate POST occurring. While the same is true when combining the Idempotent Consumer with Kafka Transactions, there are two transactional managers in play here, meaning more complexity, more things to go wrong, and more testing needed.

For each of these approaches there is nothing that can be done to completely remove the chance of a duplicate POST. It is therefore important to ensure that that call is itself idempotent, with the third party service dealing with that idempotent requirement.

Next...

In the second part of this article the different deduplication patterns are drawn out in detail for each failure scenario to show how and where duplicates occur.

Introduction

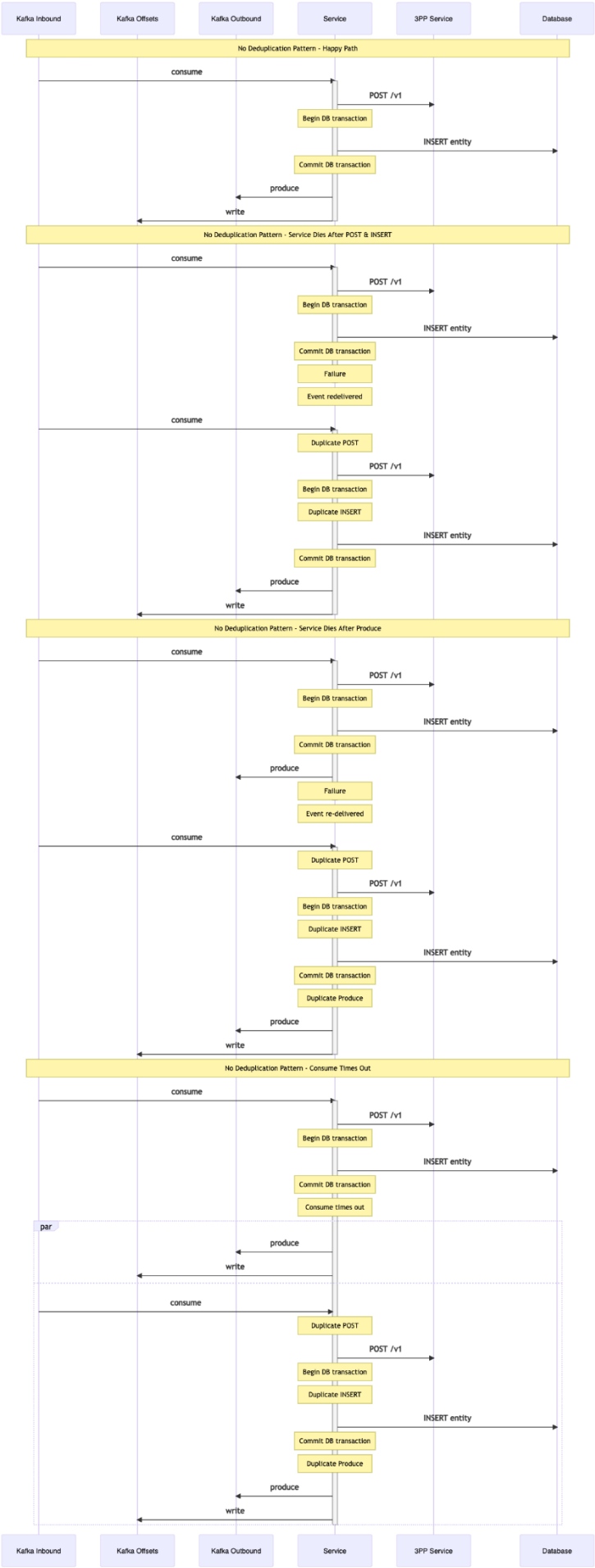
In the first part of this article the different deduplication patterns are described and the various duplicates that can occur are called out. In this part each of the failure scenarios are drawn out in detail to show how and where duplicates occur.

Diagramming The Failures

No Deduplication Pattern

The basic flow that performs a REST POST request, inserts an entity in the database, and produces a resulting event, with no deduplication patterns in place.

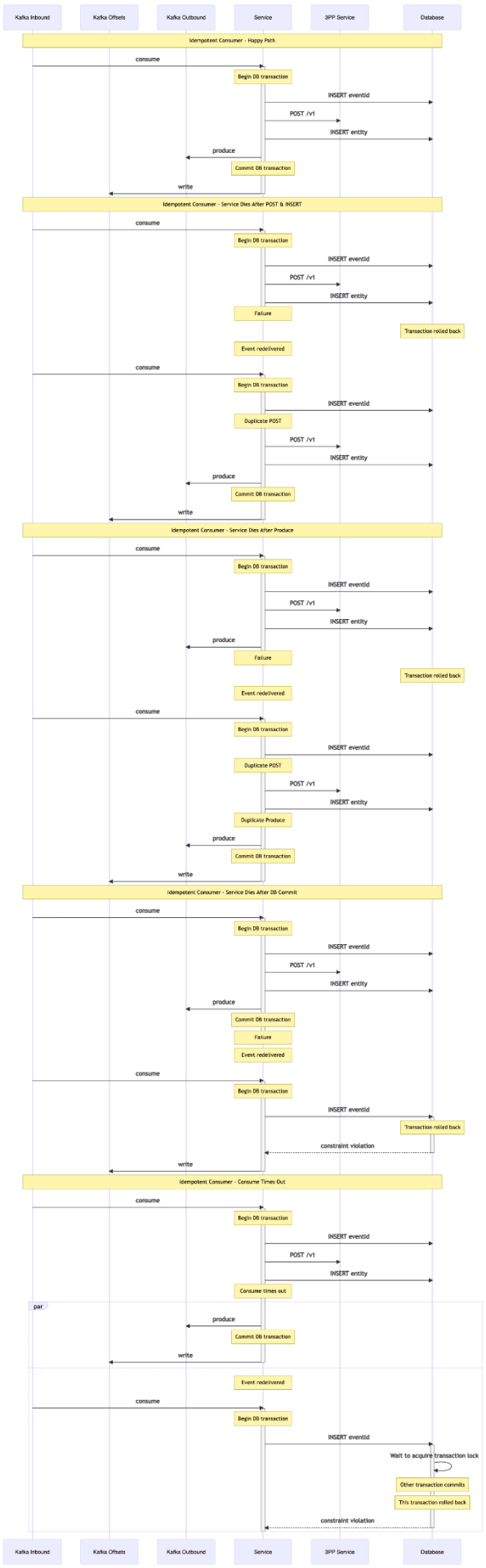
1. Consume event
2. REST POST request
3. Begin DB transaction
4. DB insert entity
5. Commit DB transaction
6. Produce event
7. Update consumer offsets


Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database
No Deduplication Pattern - Happy Path
consume
POST /v1
Begin DB transaction
INSERT entity
Commit DB transaction
produce
write
No Deduplication Pattern - Service Dies After POST & INSERT
consume
POST /v1
Begin DB transaction
INSERT entity
Commit DB transaction
Failure
Event redelivered
consume
Duplicate POST
POST /v1
Begin DB transaction
Duplicate INSERT
INSERT entity
Commit DB transaction
produce
write
No Deduplication Pattern - Service Dies After Produce
consume
POST /v1
Begin DB transaction
INSERT entity
Commit DB transaction
produce
Failure
Event re-delivered
consume
Duplicate POST
POST /v1
Begin DB transaction
Duplicate INSERT
INSERT entity
Commit DB transaction
Duplicate Produce
produce
write
No Deduplication Pattern - Consume Times Out
consume
POST /v1
Begin DB transaction
INSERT entity
Commit DB transaction
Consume times out
par
produce
write
consume
Duplicate POST
POST /v1
Begin DB transaction
Duplicate INSERT
INSERT entity
Commit DB transaction
Duplicate Produce
produce
write
Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database

Idempotent Consumer

The DB transaction must wrap the bulk of the processing, starting with inserting the eventId, and ending after the outbound event is produced. The consumer offsets update must be the last action so that any failure results in a redelivery.

1. Consume event
2. Begin DB transaction
3. DB insert eventId
4. REST POST request
5. DB insert entity
6. Produce event
7. Commit DB transaction
8. Update consumer offsets

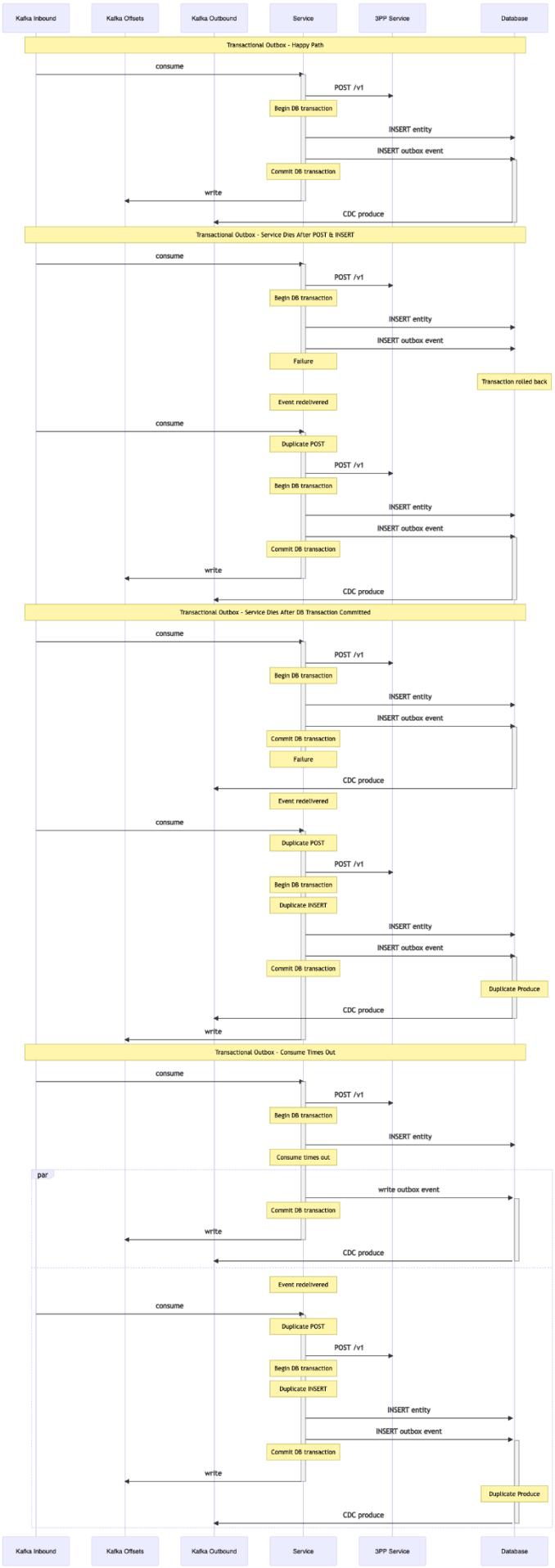

Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database
Idempotent Consumer - Happy Path
consume
Begin DB transaction
INSERT eventId
POST /v1
INSERT entity
produce
Commit DB transaction
write
Idempotent Consumer - Service Dies After POST & INSERT
consume
Begin DB transaction
INSERT eventId
POST /v1
INSERT entity
Failure
Transaction rolled back
Event redelivered
consume
Begin DB transaction
INSERT eventId
Duplicate POST
POST /v1
INSERT entity
produce
Commit DB transaction
write
Idempotent Consumer - Service Dies After Produce
consume
Begin DB transaction
INSERT eventId
POST /v1
INSERT entity
produce
Failure
Transaction rolled back
Event redelivered
consume
Begin DB transaction
INSERT eventId
Duplicate POST
POST /v1
INSERT entity
Duplicate Produce
produce
Commit DB transaction
write
Idempotent Consumer - Service Dies After DB Commit
consume
Begin DB transaction
INSERT eventId
POST /v1
INSERT entity
produce
Commit DB transaction
Failure
Event redelivered
consume
Begin DB transaction
INSERT eventId
Transaction rolled back
constraint violation
write
Idempotent Consumer - Consume Times Out
consume
Begin DB transaction
INSERT eventId
POST /v1
INSERT entity
Consume times out
par
produce
Commit DB transaction
write
Event redelivered
consume
Begin DB transaction
INSERT eventId
Wait to acquire transaction lock
Other transaction commits
This transaction rolled back
constraint violation
Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database

Transactional Outbox

The DB transaction wraps any entity writes and the insert of the outbox event. CDC results in the event being produced to the outbound topic.

1. Consume event
2. REST POST request
3. Begin DB transaction
4. DB insert entity
5. DB insert outbox event
6. Commit DB transaction
7. Update consumer offsets

- Event produced via CDC

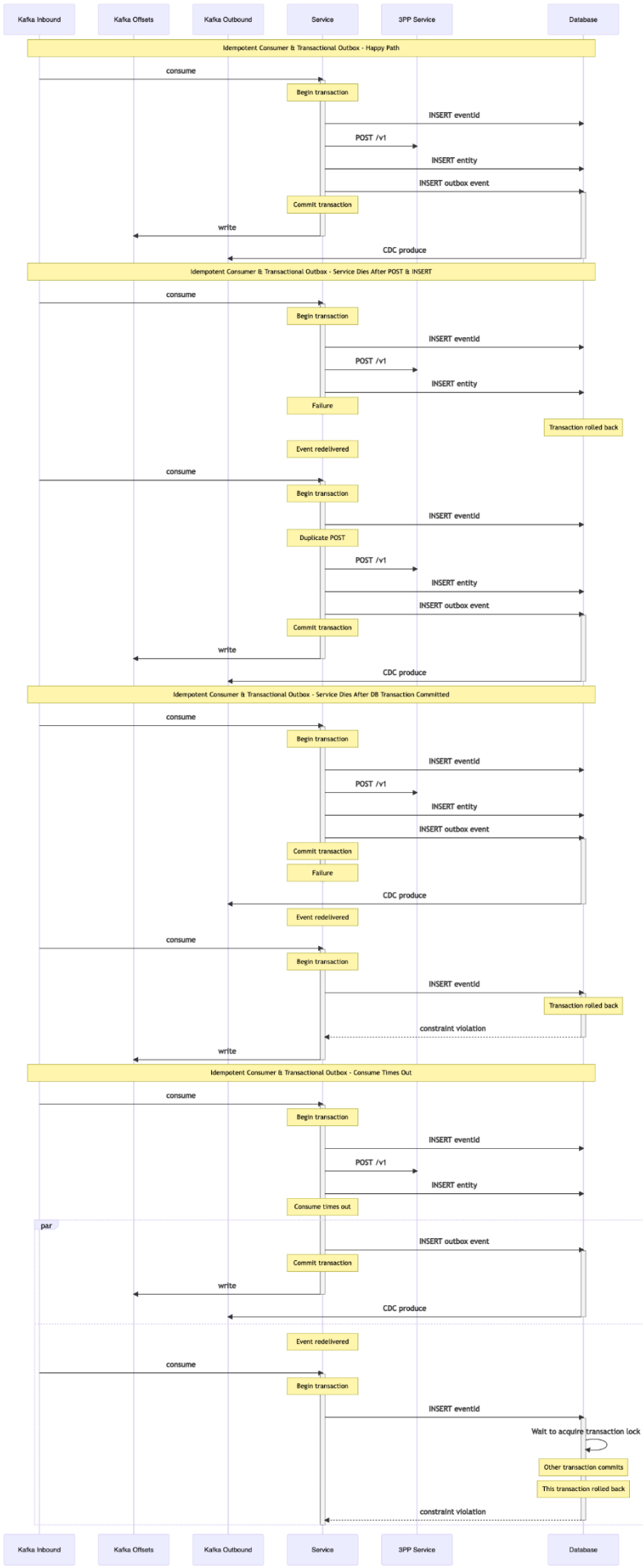

Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database
Transactional Outbox - Happy Path
consume
POST /v1
Begin DB transaction
INSERT entity
INSERT outbox event
Commit DB transaction
write
CDC produce
Transactional Outbox - Service Dies After POST & INSERT
consume
POST /v1
Begin DB transaction
INSERT entity
INSERT outbox event
Failure
Transaction rolled back
Event redelivered
consume
Duplicate POST
POST /v1
Begin DB transaction
INSERT entity
INSERT outbox event
Commit DB transaction
write
CDC produce
Transactional Outbox - Service Dies After DB Transaction Committed
consume
POST /v1
Begin DB transaction
INSERT entity
INSERT outbox event
Commit DB transaction
Failure
CDC produce
Event redelivered
consume
Duplicate POST
POST /v1
Begin DB transaction
Duplicate INSERT
INSERT entity
INSERT outbox event
Commit DB transaction
Duplicate Produce
CDC produce
write
Transactional Outbox - Consume Times Out
consume
POST /v1
Begin DB transaction
INSERT entity
Consume times out
par
write outbox event
Commit DB transaction
write
CDC produce
Event redelivered
consume
Duplicate POST
POST /v1
Begin DB transaction
Duplicate INSERT
INSERT entity
INSERT outbox event
Commit DB transaction
write
Duplicate Produce
CDC produce
Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database

Idempotent Consumer And Transactional Outbox

Combining the Idempotent Consumer and Transactional Outbox patterns. The event deduplication, any entity writes, and the outbox event write that results in the event being produced to the outbound topic are atomic.

1. Consume event
2. Begin DB transaction
3. DB insert eventId
4. REST POST request
5. DB insert entity
6. DB insert outbox event
7. Commit DB transaction
8. Update consumer offsets

- Event produced via CDC

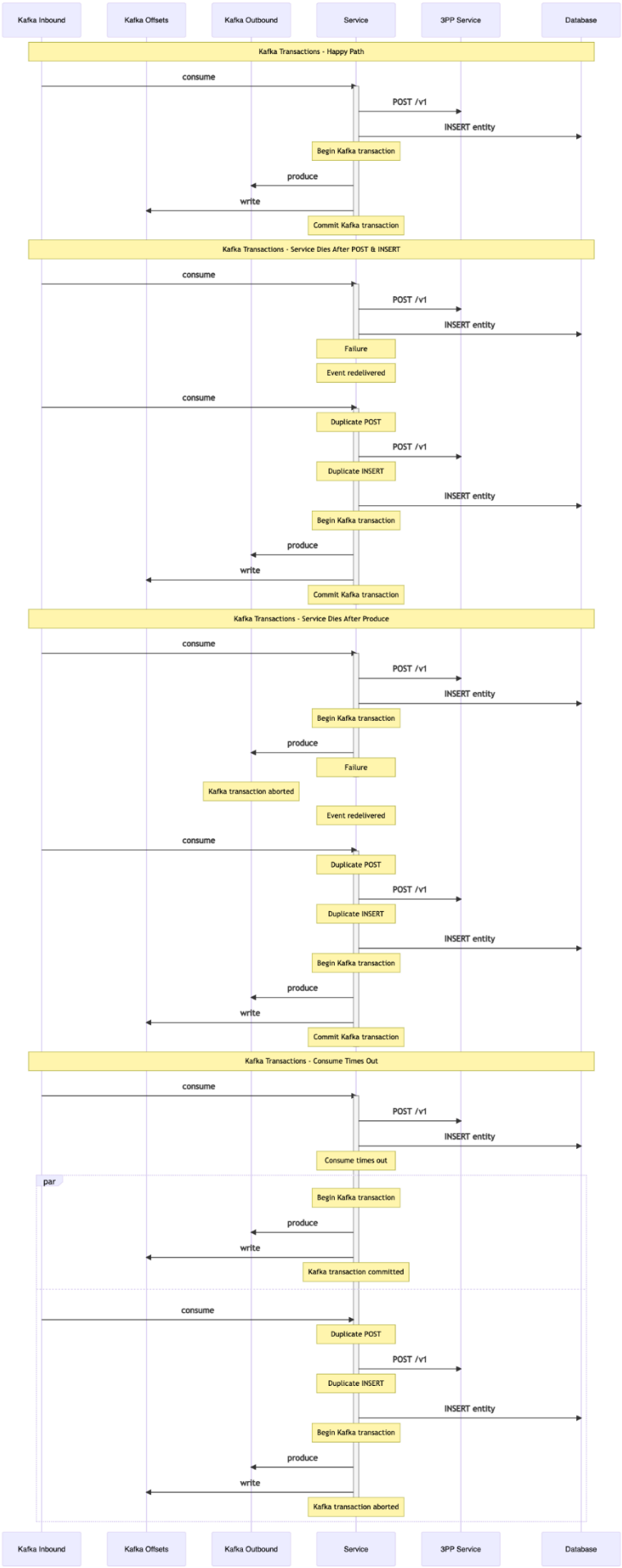

Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database
Idempotent Consumer & Transactional Outbox - Happy Path
consume
Begin transaction
INSERT eventId
POST /v1
INSERT entity
INSERT outbox event
Commit transaction
write
CDC produce
Idempotent Consumer & Transactional Outbox - Service Dies After POST & INSERT
consume
Begin transaction
INSERT eventId
POST /v1
INSERT entity
Failure
Transaction rolled back
Event redelivered
consume
Begin transaction
INSERT eventId
Duplicate POST
POST /v1
INSERT entity
INSERT outbox event
Commit transaction
write
CDC produce
Idempotent Consumer & Transactional Outbox - Service Dies After DB Transaction Committed
consume
Begin transaction
INSERT eventId
POST /v1
INSERT entity
INSERT outbox event
Commit transaction
Failure
CDC produce
Event redelivered
consume
Begin transaction
INSERT eventId
Transaction rolled back
constraint violation
write
Idempotent Consumer & Transactional Outbox - Consume Times Out
consume
Begin transaction
INSERT eventId
POST /v1
INSERT entity
Consume times out
par
INSERT outbox event
Commit transaction
write
CDC produce
Event redelivered
consume
Begin transaction
INSERT eventId
Wait to acquire transaction lock
Other transaction commits
This transaction rolled back
constraint violation
Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database

Kafka Transactions

With Kafka Transactions the publish of the outbound event and the consumer offsets update are atomic.

1. Consume event
2. REST POST request
3. Begin DB transaction
4. DB insert entity
5. Commit DB transaction
6. Begin Kafka transaction
7. Produce event
8. Update consumer offsets
9. Commit Kafka transaction

In the failure scenario where the event is produced to the topic but the transaction is aborted, the write is successful, however the event is not marked as committed on the transaction log. Consumers of this topic need to be configured as READ_COMMITTED to ensure they do not consume events that have not been successfully marked as committed.

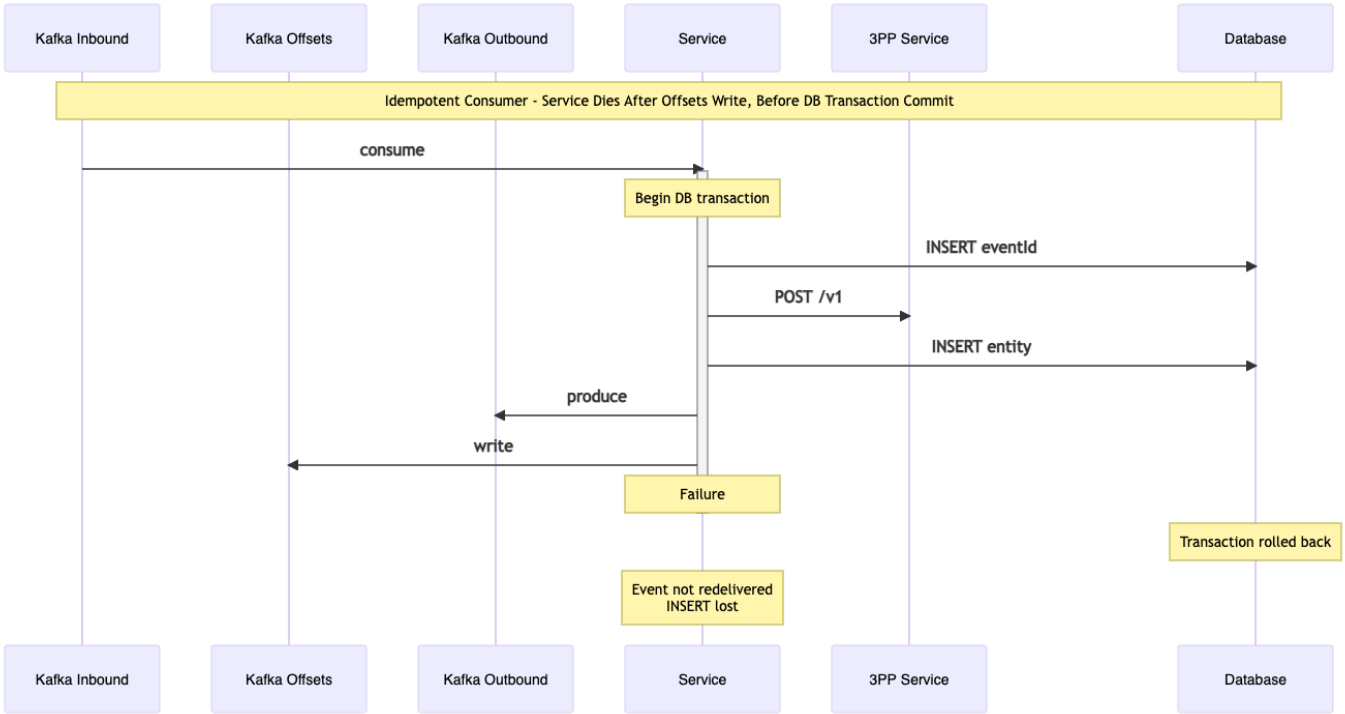

Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database
Kafka Transactions - Happy Path
consume
POST /v1
INSERT entity
Begin Kafka transaction
produce
write
Commit Kafka transaction
Kafka Transactions - Service Dies After POST & INSERT
consume
POST /v1
INSERT entity
Failure
Event redelivered
consume
Duplicate POST
POST /v1
Duplicate INSERT
INSERT entity
Begin Kafka transaction
produce
write
Commit Kafka transaction
Kafka Transactions - Service Dies After Produce
consume
POST /v1
INSERT entity
Begin Kafka transaction
produce
Failure
Kafka transaction aborted
Event redelivered
consume
Duplicate POST
POST /v1
Duplicate INSERT
INSERT entity
Begin Kafka transaction
produce
write
Commit Kafka transaction
Kafka Transactions - Consume Times Out
consume
POST /v1
INSERT entity
Consume times out
par
Begin Kafka transaction
produce
write
Kafka transaction committed
consume
Duplicate POST
POST /v1
Duplicate INSERT
INSERT entity
Begin Kafka transaction
produce
write
Kafka transaction aborted
Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database

Appendix: Avoiding Data Loss

Not all patterns can be combined, and the patterns that are applied impact the order that the actions can happen in order to ensure that no data (event or database write) can be lost. This section illustrates a number of flows showing how data loss could occur for these situations.

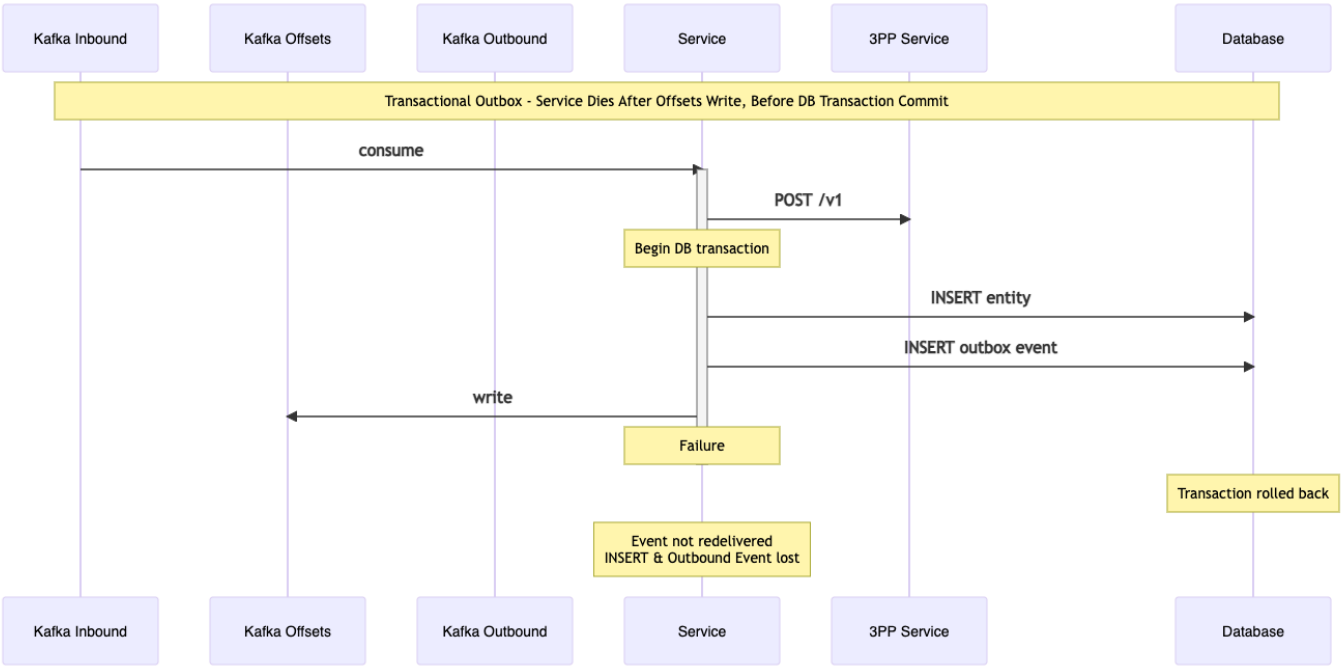
Idempotent Consumer

With the Idempotent Consumer pattern the database transaction must follow the event publish. If it precedes it, and the service fails before the database transaction is committed, then the database writes will be lost as the event has been marked as consumed and will not be redelivered.



Transactional Outbox

With the Transactional Outbox pattern the database transaction must precede the consumer offsets write. If the consumer offsets are updated, marking the event as consumed, and the service dies before the database transaction is committed, then the event will not be redelivered and the database writes and outbound event publish will be lost.

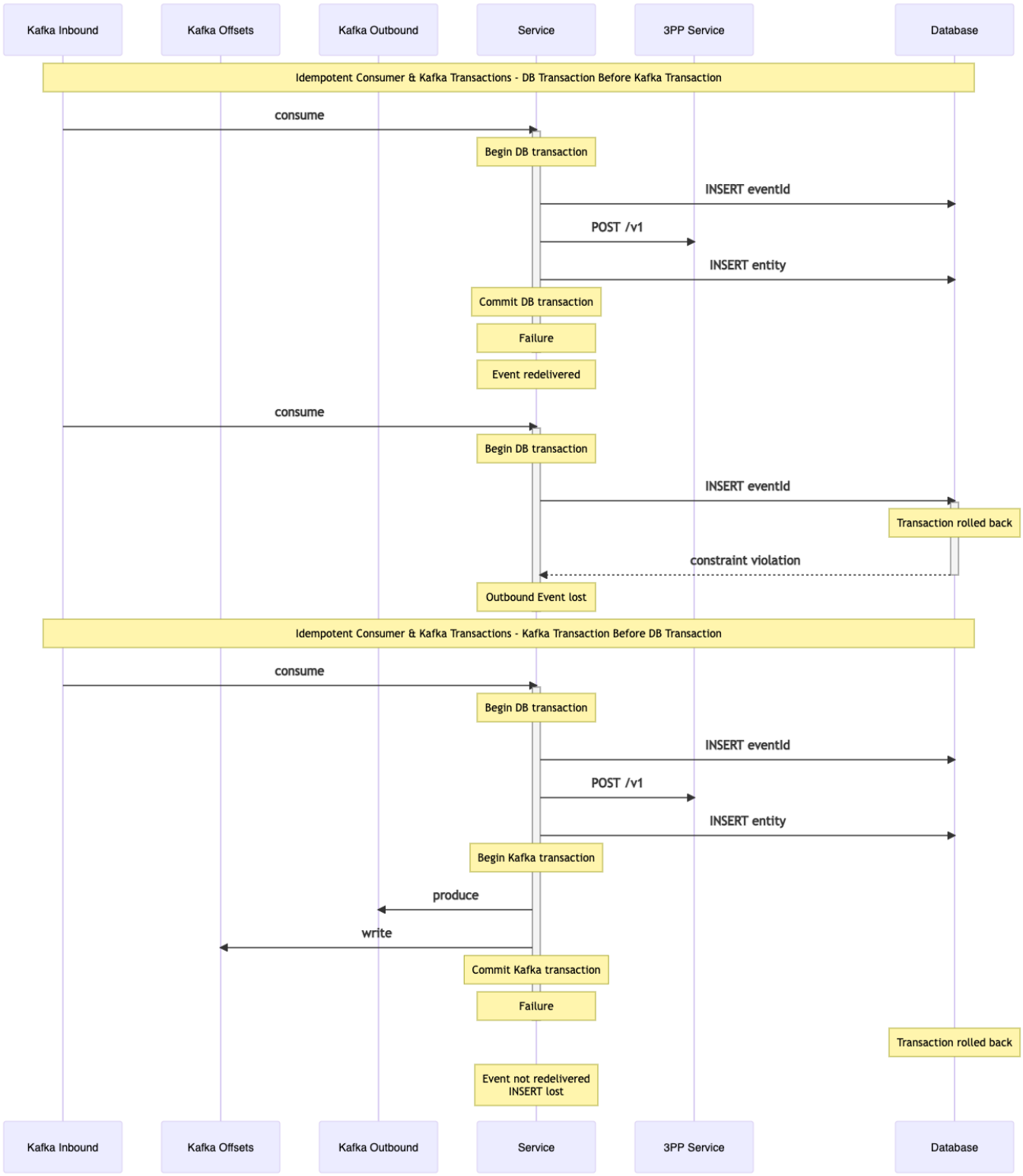


Idempotent Consumer And Kafka Transactions

It is not possible to combine the Idempotent Consumer and Kafka Transactions patterns without risking data loss in certain failure scenarios as the following diagram illustrates.

If the database transaction commit precedes the Kafka transaction commit, and the service fails before the Kafka transaction is committed, then when the event is redelivered it will be deduplicated by the Idempotent Consumer. This means the resulting outbound event will never be published.

If the Kafka transaction commit precedes the database transaction commit, and the service fails before the database transaction is committed, then the event will not be redelivered as the Kafka transaction has updated the consumer offsets. This means any database writes will be lost.


Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database
Idempotent Consumer & Kafka Transactions - DB Transaction Before Kafka Transaction
consume
Begin DB transaction
INSERT eventId
POST /v1
INSERT entity
Commit DB transaction
Failure
Event redelivered
consume
Begin DB transaction
INSERT eventId
Transaction rolled back
constraint violation
Outbound Event lost
Idempotent Consumer & Kafka Transactions - Kafka Transaction Before DB Transaction
consume
Begin DB transaction
INSERT eventId
POST /v1
INSERT entity
Begin Kafka transaction
produce
write
Commit Kafka transaction
Failure
Transaction rolled back
Event not redelivered
INSERT lost
Kafka Inbound
Kafka Offsets
Kafka Outbound
Service
3PP Service
Database